The Globe oposiciones

Bloque 3 - Tema 6 ejercicios 2

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS

Soluciones

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

Enunciado 1

Explique detalladamente qué hace y la salida que imprime por pantalla el siguiente código:

```
public class Empleado {
      static void imprimir() {System.out.println("Empleado");} }

public class Interino extends Empleado {
      static void imprimir() {System.out.println("Interino");} }

public class Recursos {
      public static void main(String[] args) {
            Empleado.imprimir();
            Interino.imprimir();
            Empleado e = new Interino();
            e.imprimir();
      }
   }
```

SOLUCIÓN

Se han implementado tres clases:

- Empleado: tiene un método estático imprimir que muestra por pantalla la cadena Empleado. El constructor (implícito) es el constructor por defecto.
- Interino: clase que extiende de Empleado. Sobreescribe/redefine el método estático imprimir para que muestre por pantalla la cadena Interino.
- Recursos: clase que contiene el método main, por donde comenzará la ejecución.

El concepto clave es la ligadura estática, es decir, en tiempo de compilación se comprueba para el objeto la clase sobre la que ha sido declarado.

Traza de la ejecución del método main:

(1) Se llama al método imprimir de Empleado.

(2) Se llama al método imprimir de Interino.

(3) Se declara el objeto e del tipo Empleado y se instancia como un objeto de Interino.

(4) Se llama al método imprimir de Empleado pues no importa el tipo de objeto e sino el tipo con el que se declaró.

Por tanto, la salida por pantalla es:
Empleado
Interino
Empleado

Enunciado 2

Defina una clase Linea con dos atributos: puntoA y_puntoB. Son dos puntos por los que pasa la línea en un espacio de dos dimensiones. La clase dispondrá de los siguientes métodos:

- Linea(): Constructor predeterminado que crea una línea con sus dos puntos como (0,0) y (0,0).

- Linea(Punto, Punto): Constructor que recibe como parámetros dos objetos de la clase Punto, que son utilizados para inicializar los atributos.

- mueveDerecha(double): Desplaza la línea a la derecha la distancia que se indique.

- mueveIzquierda(double): Desplaza la línea a la izquierda la distancia que se indique.

- mueveArriba(double): Desplaza la línea hacia arriba la distancia que se indique.

- mueveAbajo(double): Desplaza la línea hacia abajo la distancia que se indique.

- Getters y setters.

- Método que nos permita mostrar la información de la línea de la forma [puntoA,puntoB]. Por ejemplo: [(0.0,0.0),(1.0,1.0)].

SOLUCIÓN

```
public class Punto {
    private double x;
    private double y;

    public Punto() {
        x = 0;
        y = 0;
    }

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return y;
    }
```

```
    public void setY(double y) {
        this.y = y;
    }
}


public class Linea {
    private Punto puntoA;
    private Punto puntoB;

    public Linea() {
        puntoA = new Punto();
        puntoB = new Punto();
    }

    public Linea(Punto p1, Punto p2) {
        puntoA = new Punto(p1.getX(), p1.getY());
        puntoB = new Punto(p2.getX(), p2.getY());
    }

    public void mueveDerecha(double d) {
        puntoA.setX(puntoA.getX()+d);
        puntoB.setX(puntoB.getX()+d);
    }

    public void mueveIzquierda(double d) {
        puntoA.setX(puntoA.getX()-d);
        puntoB.setX(puntoB.getX()-d);
    }

    public void mueveArriba(double d) {
        puntoA.setY(puntoA.getY()+d);
        puntoB.setY(puntoB.getY()+d);
    }

    public void mueveAbajo(double d) {
        puntoA.setY(puntoA.getY()-d);
        puntoB.setY(puntoB.getY()-d);
    }

    public Punto getPuntoA() {
        return puntoA;
    }

    public Punto getPuntoB() {
        return puntoB;
    }

    public void setPuntoA(Punto puntoA) {
        this.puntoA = puntoA;
    }

    public void setPuntoB(Punto puntoB) {
        this.puntoB = puntoB;
    }

    @Override
    public String toString() {
```

```
        return "[("+puntoA.getX() + ", " + puntoA.getY() + "), (" +
                puntoB.getX() + ", " + puntoB.getY() + ")]";
    }
}

public class PruebaLinea {
    public static void main(String[] args) {
        Punto p1 = new Punto(1,2);
        Punto p2 = new Punto(5,2);

        Linea lin1 = new Linea(p1,p2);

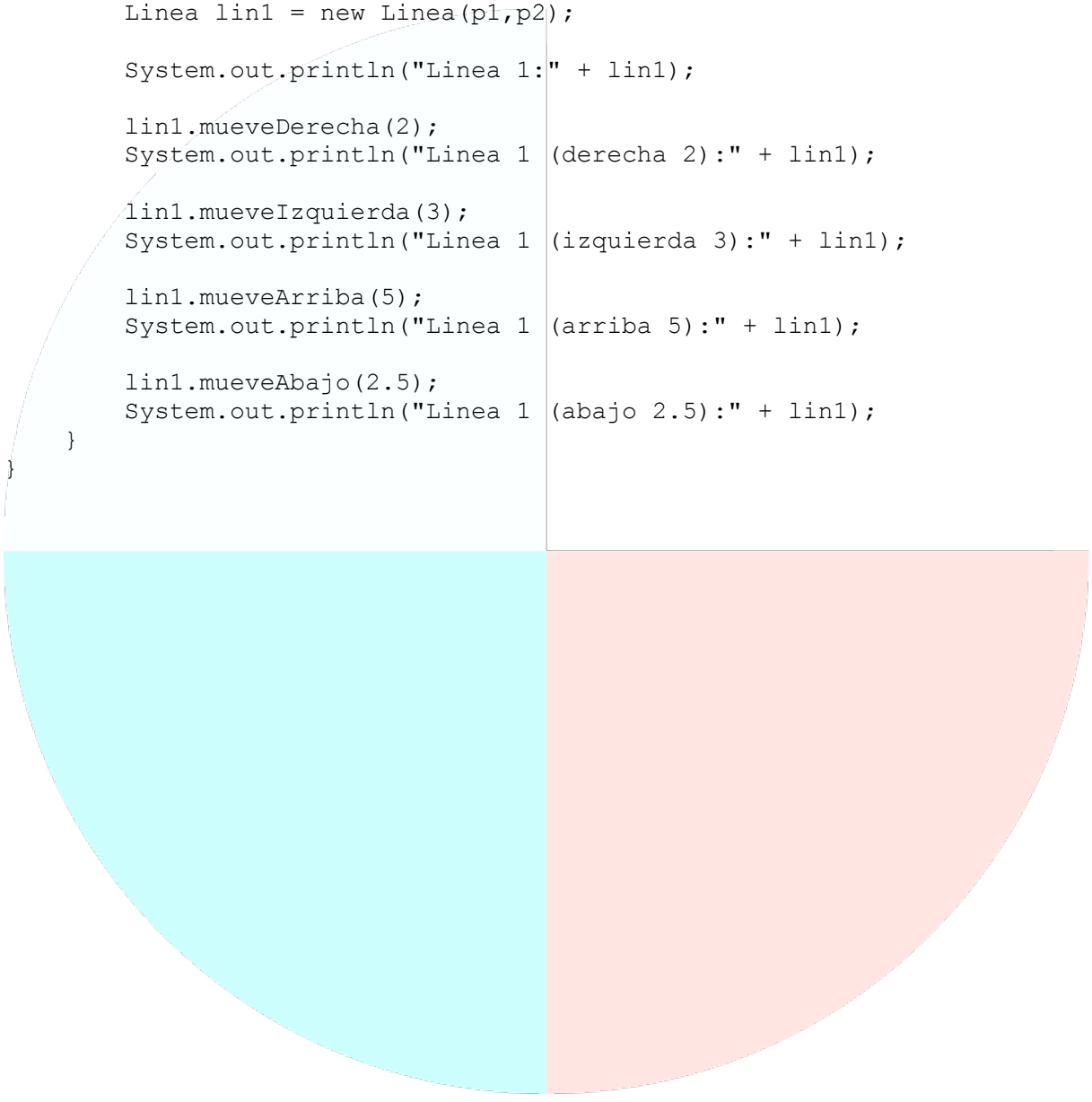
        System.out.println("Linea 1:" + lin1);

        lin1.mueveDerecha(2);
        System.out.println("Linea 1 (derecha 2):" + lin1);

        lin1.mueveIzquierda(3);
        System.out.println("Linea 1 (izquierda 3):" + lin1);

        lin1.mueveArriba(5);
        System.out.println("Linea 1 (arriba 5):" + lin1);

        lin1.mueveAbajo(2.5);
        System.out.println("Linea 1 (abajo 2.5):" + lin1);
    }
}
```



Enunciado 3

Desarrolle una clase Cafetera con atributos_capacidadMaxima (la cantidad máxima de café que puede contener la cafetera) y _cantidadActual (la cantidad actual de café que hay en la cafetera). Implemente, al menos, los siguientes métodos:

- Constructor predeterminado: establece la capacidad máxima en 1000 (c.c.) y la actual en cero (cafetera vacía).
- Constructor con la capacidad máxima de la cafetera; inicializa la cantidad actual de café igual a la capacidad máxima.
- Constructor con la capacidad máxima y la cantidad actual. Si la cantidad actual es mayor que la capacidad máxima de la cafetera, la ajustará al máximo.
- Getters y setter.
- llenarCafetera(): pues eso, hace que la cantidad actual sea igual a la capacidad.
- servirTaza(int): simula la acción de servir una taza con la capacidad indicada. Si la cantidad actual de café "no alcanza" para llenar la taza, se sirve lo que quede.
- vaciarCafetera(): pone la cantidad de café actual en cero.
- agregarCafe(int): añade a la cafetera la cantidad de café indicada.

SOLUCIÓN

```
public class Cafetera {
    private int capacidadMaxima;
    private int cantidadActual;

    public Cafetera() {
        capacidadMaxima = 1000;
        cantidadActual = 0;
    }

    public Cafetera(int cm) {
        capacidadMaxima = cm;
        cantidadActual = cm;
    }

    public Cafetera(int cm, int ca) {
        capacidadMaxima = cm;
        cantidadActual = ca;
        if(ca>cm)
            cantidadActual = capacidadMaxima;
    }

    public int getCapacidadMaxima() {
        return capacidadMaxima;
    }
```

```
    public int getCantidadActual() {
        return cantidadActual;
    }

    public void setCapacidadMaxima(int capacidadMaxima) {
        this.capacidadMaxima = capacidadMaxima;
    }

    public void setCantidadActual(int cantidadActual) {
        this.cantidadActual = cantidadActual;
    }

    public void llenarCafetera() {
        cantidadActual = capacidadMaxima;
    }

    public void servirTaza(int cantidad) {
        if(cantidadActual-cantidad >= 0)
            cantidadActual -= cantidad;
        else
            cantidadActual = 0;
    }

    public void vaciarCafetera() {
        cantidadActual = 0;
    }

    public void agregarCafe(int cantidad) {
        if(cantidadActual + cantidad <= capacidadMaxima)
            cantidadActual += cantidad;
        else
            cantidadActual = capacidadMaxima;
    }
}
```

Enunciado 4

Defina una clase abstracta Cuenta.

- **Atributos:**
  - numerocuenta: entero largo.
  - saldo: double
  - cliente: atributo de la clase Persona (que tiene nombre y apellidos, y NIF).
- **Y con los siguientes métodos:**
  - Constructor parametrizado que recibe un cliente y un número de cuenta.
  - Getters para los tres atributos.
  - ingresar(double): permitirá ingresar una cantidad en la cuenta.
  - abstract retirar(double): permitirá sacar una cantidad de la cuenta (si hay saldo).
  - actualizarSaldo(): actualizará el saldo de la cuenta, pero cada cuenta lo hace de una forma diferente.

Defina las subclases de Cuenta que se describen a continuación:

- CuentaCorriente: Cuenta normal con un interés fijo del 1.5%. Incluir constructor parametrizado y método toString().
- CuentaAhorro: Esta cuenta tiene como atributos el interés variable a lo largo del año y un saldo mínimo necesario. Al retirar dinero hay que tener en cuenta que no se sobrepase el saldo mínimo. Incluir constructor parametrizado, método toString() y método para cambiar el interés.

Crea un programa que cree varias cuentas y pruebe sus características.

SOLUCIÓN

```
public class Persona {
    private String nombre;
    private String apellidos;
    private String nif;

    public Persona() {
    }

    public Persona(String nombre, String apellidos, String nif) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.nif = nif;
    }

    public String getNombre() {
```

```
        return nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public String getNif() {
        return nif;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    public void setNif(String nif) {
        this.nif = nif;
    }

    @Override
    public String toString() {
        return "Persona{" + "nombre=" + nombre + ", apellidos=" +
                apellidos + ", nif=" + nif + '}';
    }
}
```

```
public abstract class Cuenta {
    private long numeroCuenta;
    protected double saldo;
    private Persona cliente;

    public Cuenta() {
    }

    public Cuenta(long numeroCuenta, Persona cliente) {
        this.numeroCuenta = numeroCuenta;
        this.cliente = cliente;
    }

    public long getNumeroCuenta() {
        return numeroCuenta;
    }

    public double getSaldo() {
        return saldo;
    }

    public Persona getCliente() {
        return cliente;
    }

    public void ingresar(double cantidad) {
        saldo += cantidad;
```

```
    }

    public abstract void retirar(double cantidad);

    public abstract void actualizarSaldo();

    @Override
    public String toString() {
        return "Cuenta{" + "numeroCuenta=" + numeroCuenta + ",
cliente=" + cliente + '}';
    }
}


public class CuentaCorriente extends Cuenta {
    private final double interesFijo = 0.015;

    public CuentaCorriente(long numeroCuenta, Persona cliente) {
        super(numeroCuenta, cliente);
    }

    @Override
    public void retirar(double cantidad) {
        if(saldo >= cantidad)
           saldo -= cantidad;
    }

    @Override
    public void actualizarSaldo() {
        saldo = saldo + (saldo*interesFijo);
    }

    @Override
    public String toString() {
        return super.toString() + "CuentaCorriente{" + "N° cuenta=" +
                      getNumeroCuenta() + ", interesFijo=" +
                      interesFijo + ", Saldo=" + getSaldo() + "}";
    }
}


public class CuentaAhorro extends Cuenta {
    private double interes;
    private double saldoMinimo;

    public CuentaAhorro(double interes, double saldoMinimo, long
numeroCuenta, Persona cliente) {
        super(numeroCuenta, cliente);
        this.interes = interes;
        this.saldoMinimo = saldoMinimo;
    }

    public double getInteres() {
        return interes;
    }

    public void setInteres(double interes) {
        this.interes = interes;
```

```
    }

    @Override
    public void retirar(double cantidad) {
        if((saldo-cantidad) >= saldoMinimo)
           saldo -= cantidad;
    }

    @Override
    public void actualizarSaldo() {
        saldo = saldo + (saldo*interes);
    }

    @Override
    public String toString() {
        return super.toString() + "CuentaAhorro{" + "N° cuenta=" +
                        getNumeroCuenta() + ", interes=" + interes +
                        ", Saldo=" + getSaldo() + ", saldoMinimo=" +
                        saldoMinimo + "}";
    }
}


public class PruebaCuenta {
    public static void main(String[] args) {
        Cuenta c1 = new CuentaCorriente(1234, new
Persona("Pablo","Arellano","1234567P"));
        Cuenta c2 = new CuentaAhorro(0.04,200,2345, new
           Persona("Alex","Mirasol","2345678X"));

        System.out.println("c1:" + c1);
        System.out.println("c2:" + c2);

        c1.ingresar(500);
        c2.ingresar(1000);

        System.out.println("c1:" + c1);
        System.out.println("c2:" + c2);

        c1.actualizarSaldo();
        c2.actualizarSaldo();

        System.out.println("c1:" + c1);
        System.out.println("c2:" + c2);

        c1.retirar(100);
        c2.retirar(300);

        System.out.println("c1:" + c1);
        System.out.println("c2:" + c2);

        c2.retirar(600);

        System.out.println("c2:" + c2);
    }
}
```

Enunciado 5

Defina los siguientes elementos:

- interface Puerta: con los métodos abrir y cerrar.
- interface PuertaBloqueable: derivada de Puerta, con los métodos bloquea y desbloquea.
- interface Alarma: con los métodos isAlarmaActivada, activarAlarma y desactivarAlarma.
- clase ComponentedeCoche: con los atributos descripción, peso y coste, y un método que muestre dichos atributos.
- clase PuertaCoche, con el atributo boolean estaBloqueada, y que extienda la clase ComponentedeCoche e implemente las interfaces Alarma y Puertabloqueable.

Escriba un programa que pruebe la clase PuertaCoche y todas las operaciones que admite.

SOLUCIÓN

```
public interface Puerta {
    public void abrir();
    public void cerrar();
}

public interface PuertaBloqueable extends Puerta {
    public void bloquea();
    public void desbloquea();
}

public interface Alarma {
    public boolean isAlarmaActivada();
    public void activarAlarma();
    public void desactivarAlarma();
}

public class ComponenteDeCoche {
    private String descripcion;
    private double peso;
    private double coste;

    public ComponenteDeCoche() {
    }

    public ComponenteDeCoche(String descripcion, double peso,
                double coste) {
        this.descripcion = descripcion;
        this.peso = peso;
        this.coste = coste;
    }

    @Override
```

```
    public String toString() {
        return "ComponenteDeCoche{" + "descripcion=" + descripcion +
                ", peso=" + peso + ", coste=" + coste + '}';
    }
}


public class PuertaCoche extends ComponenteDeCoche
        implements PuertaBloqueable, Alarma {
    private boolean estaBloqueada;
    private boolean alarmaActivada;

    public PuertaCoche(String descripcion, double peso, double coste) {
        super(descripcion, peso, coste);
    }

    public boolean isEstaBloqueada() {
        return estaBloqueada;
    }

    public void setEstaBloqueada(boolean estaBloqueada) {
        this.estaBloqueada = estaBloqueada;
    }

    @Override
    public void bloquea() {
        if(!isAlarmaActivada())
           activarAlarma();
    }

    @Override
    public void desbloquea() {
        if(isAlarmaActivada())
            desactivarAlarma();
    }

    @Override
    public void abrir() {
        if(estaBloqueada)
            desbloquea();
        estaBloqueada = false;
    }

    @Override
    public void cerrar() {
        bloquea();
        estaBloqueada = true;
    }

    @Override
    public boolean isAlarmaActivada() {
        return alarmaActivada;
    }

    @Override
    public void activarAlarma() {
        alarmaActivada = true;
    }
```

```
    @Override
    public void desactivarAlarma() {
        alarmaActivada = false;
    }

    @Override
    public String toString() {
        return super.toString() + "PuertaCoche{" +
           "estaBloqueada=" + estaBloqueada + ", alarmaActivada=" +
           alarmaActivada + '}';
    }


public class PruebaPuertaCoche {
    public static void main(String[] args) {
        PuertaCoche p1 = new PuertaCoche("Puerta 1",15,100);
        PuertaCoche p2 = new PuertaCoche("Puerta 2",15,105);

        System.out.println("p1: " +p1);
        System.out.println("p2: " +p2);

        p1.abrir();
        p2.abrir();

        System.out.println("Despues de abrir");
        System.out.println("p1: " +p1);
        System.out.println("p2: " +p2);

        p1.cerrar();
        p2.cerrar();

        System.out.println("Despues de cerrar");
        System.out.println("p1: " +p1);
        System.out.println("p2: " +p2);

        p1.abrir();
        p2.cerrar();

        System.out.println("Despues de abrir p1 y cerrar p2");
        System.out.println("p1: " +p1);
        System.out.println("p2: " +p2);
    }
}
```